🔹 Django Middleware কী?

Middleware হলো Django-এর এমন একটি সিস্টেম যেটা **request আর response এর মাঝখানে কাজ করে**।

সহজভাবে বললে 👉
Browser → **Middleware** → View → **Middleware** → Browser

মানে, যখন user কোনো request পাঠায়, সেটা প্রথমে middleware গুলোর মধ্য দিয়ে যায়।
আবার view থেকে response বের হলে সেটাও middleware হয়ে user-এর কাছে যায়।

🔹 Middleware কোথায় থাকে?

তুমি `settings.py` ফাইলে এটা দেখবে:

```
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
]
```

এগুলো built-in middleware।

🔹 Middleware কিভাবে কাজ করে?

Middleware দুইভাবে কাজ করে:

1. **Request প্রসেস করে (before view)**
2. **Response প্রসেস করে (after view)**

Flow Diagram:

```
Request →
   Middleware 1 →
   Middleware 2 →
   View →
   Middleware 2 →
   Middleware 1 →
Response
```

👉 উপরের দিক দিয়ে request যায়
👉 নিচের দিক দিয়ে response ফিরে আসে

🔹 একটি সহজ Custom Middleware Example

ধরো তুমি সব request log করতে চাও।

Step 1: middleware file তৈরি করো

`your_app/middleware.py`

```
class SimpleMiddleware:

    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        print("Before View")

        response = self.get_response(request)

        print("After View")

        return response
```

Step 2: settings.py তে add করো

```
MIDDLEWARE = [
    ...
    'your_app.middleware.SimpleMiddleware',
]
```

এখন কী হবে?

যখন কোনো request আসবে:

1. `"Before View"` print হবে
2. View execute হবে
3. `"After View"` print হবে

🔹 Built-in গুরুত্বপূর্ণ Middleware

1️⃣ Security Middleware

👉 HTTP security header add করে

```
'django.middleware.security.SecurityMiddleware'
```

2️⃣ CSRF Middleware

👉 Form submit protection দেয়

```
'django.middleware.csrf.CsrfViewMiddleware'
```

3️⃣ Authentication Middleware

👉 `request.user` enable করে

```
'django.contrib.auth.middleware.AuthenticationMiddleware'
```

🔹 Middleware কখন ব্যবহার করবো?

✔ Authentication check
✔ Logging
✔ Rate limiting
✔ Custom header add করা
✔ API request modify করা
✔ IP blocking

🔹 Middleware vs View Difference

Middleware	View
সব request এ কাজ করে	নির্দিষ্ট URL এ কাজ করে
Request & Response দুটোতেই কাজ করে	শুধু logic handle করে
Global level control	Specific endpoint control

🔥 তোমার Django + DRF প্রজেক্টে Middleware কোথায় useful?

তুমি যেহেতু ML inference server / API বানাচ্ছো, middleware ব্যবহার করতে পারো:

- API request logging
- Suspicious IP block
- Custom token validation
- Performance timing

🔥 Django Middleware দিয়ে কী কী কাজ করা যায়? আর কেন ব্যবহার করবো?

Middleware হলো এমন একটা জায়গা যেখানে তুমি **সব request আর response একসাথে control করতে পারো**।

Browser → Middleware → View → Middleware → Browser

✅ Middleware দিয়ে কী কী কাজ করা যায়?

1️⃣ Authentication / User Check

Django এ `AuthenticationMiddleware` ব্যবহার করে `request.user` পাওয়া যায়।

👉 তুমি custom middleware বানিয়ে করতে পারো:

- Login না করলে block
- Admin ছাড়া access deny
- Custom token check

কেন?

✔ সব API এক জায়গা থেকে secure করা যায়
✔ প্রতিটা view এ আলাদা check লিখতে হয় না

2️⃣ Logging (সব request track করা)

```
print(request.method, request.path)
```

কেন?

✔ কে কোন API call করছে জানা যায়
✔ Debugging সহজ হয়
✔ Production monitoring করা যায়

তোমার ML API server এ এটা খুব useful হবে।

3️⃣ Rate Limiting (Spam control)

👉 একই IP থেকে বেশি request এলে block করা যায়।

কেন?

✔ DDoS protection
✔ API abuse বন্ধ করা
✔ Server overload prevent করা

4️⃣ Custom Header Add করা

```
response["X-App-Version"] = "1.0"
```

কেন?

✔ API metadata পাঠানো
✔ Security header add করা
✔ Frontend communication improve করা

5️⃣ IP Block করা

```
if request.META['REMOTE_ADDR'] == "192.168.0.1":
    return HttpResponse("Blocked")
```

কেন?

✔ Suspicious user block
✔ Admin restriction

✔ Geo restriction

6️⃣ Performance Monitoring

```
import time
start = time.time()
response = self.get_response(request)
print(time.time() - start)
```

কেন?

✔ কোন API slow বোঝা যায়
✔ Optimization করা যায়
✔ ML inference time measure করা যায়

7️⃣ Exception Handle করা (Global Error Handling)

যদি view এ error হয়, middleware সেটা catch করতে পারে।

কেন?

✔ সুন্দর error response দেওয়া যায়
✔ Crash prevent করা যায়

🧠 তাহলে Middleware কেন ব্যবহার করবো?

Problem	Middleware দিয়ে Solution
সব view এ বারবার একই code	এক জায়গায় লিখলেই হবে
Security issue	Central control
Logging দরকার	Global tracking
API slow	Performance check
Abuse হচ্ছে	Rate limiting

🎯 তোমার ক্ষেত্রে (ML + DRF project)

তুমি middleware ব্যবহার করতে পারো:

- API request logging
- Inference time calculate
- Token validation
- Suspicious IP block
- Custom analytics

🔥 Middleware না ব্যবহার করলে কী হবে?

তাহলে তোমাকে প্রতিটা view এ আলাদা আলাদা করে code লিখতে হবে।

Example:

```
def my_view(request):
    if not request.user.is_authenticated:
        return HttpResponse("Login required")
```

এভাবে ৫০টা view হলে ৫০ জায়গায় একই code 😅

Middleware দিলে একবারেই সব control করা যায়।

দারুণ 👍 এখন তোমাকে **real-world production level middleware example** দেখাই — যেটা তোমার Django + DRF + ML API প্রজেক্টে সত্যিই কাজে লাগবে।

আমি ৩টা useful middleware দেখাবো:

1. ✅ API Request Logging
2. ✅ Inference Time Tracking (ML API এর জন্য)
3. ✅ Suspicious IP Block

✅ 1️⃣ Production Level Request Logging Middleware

🎯 Problem

তুমি জানতে চাও:

- কে কোন API call করছে?
- কত সময় লাগছে?
- Response status কত?

📁 `core/middleware.py`

```
import time
import logging

logger = logging.getLogger(__name__)

class APILoggingMiddleware:

    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        start_time = time.time()

        response = self.get_response(request)

        duration = time.time() - start_time

        logger.info(
            f"User: {request.user} | "
            f"Method: {request.method} | "
            f"Path: {request.path} | "
            f"Status: {response.status_code} | "
            f"Time: {duration:.3f}s"
        )

        return response
```

---

settings.py এ add করো

```
MIDDLEWARE = [
    ...
    'core.middleware.APILoggingMiddleware',
]
```

🔥 কেন এটা powerful?

✔ Production monitoring
✔ Slow API detect করা

✔ ML inference time measure করা
✔ Debugging সহজ

✅ 2️⃣ ML Inference Time Tracking (Real Use Case)

ধরো তোমার `/predict/` API আছে।

আমরা শুধু ML endpoint track করবো।

```
class MLTimingMiddleware:

    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        if request.path.startswith('/api/predict/'):
            start = time.time()
            response = self.get_response(request)
            duration = time.time() - start

            print(f"ML Inference took {duration:.3f} seconds")

            return response

        return self.get_response(request)
```

🎯 কেন দরকার?

তুমি PyTorch model ব্যবহার করছো (remember 😉)

✔ কোন model slow বুঝবে
✔ GPU properly কাজ করছে কিনা বুঝবে
✔ Production performance optimize করা যাবে

✅ 3️⃣ Suspicious IP Block Middleware

Production API হলে এটা খুব important।

```
from django.http import JsonResponse

BLOCKED_IPS = ["192.168.1.10"]
```

```python
class BlockIPMiddleware:

    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        ip = request.META.get('REMOTE_ADDR')

        if ip in BLOCKED_IPS:
            return JsonResponse(
                {"error": "Access Denied"},
                status=403
            )

        return self.get_response(request)
```

🎯 কেন দরকার?

✔ API abuse বন্ধ
✔ Suspicious traffic block
✔ Security improve

🧠 Real Production Architecture

Browser
⬇
Middleware (Security, Logging, IP block, Timing)
⬇
View (Business Logic)
⬇
Response

🔥 Middleware vs DRF Permission (Important)

Middleware	DRF Permission
সব request এ কাজ করে	শুধু DRF view এ কাজ করে
Global control	Endpoint specific
Performance measure করা যায়	না

🚀 তোমার ML Surveillance Project এ কী করা উচিত?

আমি recommend করবো:

1. ✅ Logging Middleware
2. ✅ Inference Timing
3. ✅ Security Middleware
4. ✅ Rate limiting (DRF throttle)

এগুলো দিলে তোমার project production-ready level হবে।

ভালো 👍 এখন আমি তোমাকে **আরও real-world middleware use cases** দেখাচ্ছি — যেগুলো বড় প্রজেক্টে সত্যি ব্যবহার হয়, বিশেষ করে তুমি যেহেতু Django + DRF + ML API নিয়ে কাজ করছো।

🔥 1️⃣ Global Maintenance Mode

ধরো server update চলছে — সবাইকে temporarily block করতে চাও।

```
from django.http import JsonResponse
from django.conf import settings

class MaintenanceMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        if getattr(settings, "MAINTENANCE_MODE", False):
            return JsonResponse(
                {"message": "Server under maintenance"},
                status=503
            )

        return self.get_response(request)
```

কেন useful?

✔ Deploy এর সময় safe
✔ Database migration এর সময় user block
✔ Production control

🔥 2️⃣ Request Size Limit (ML API এর জন্য Important)

ধরো কেউ 5GB video upload করে দিল 😅

```
class RequestSizeLimitMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        max_size = 10 * 1024 * 1024  # 10MB

        if request.META.get('CONTENT_LENGTH'):
            if int(request.META['CONTENT_LENGTH']) > max_size:
                return JsonResponse(
                    {"error": "File too large"},
                    status=413
                )

        return self.get_response(request)
```

কেন?

✔ Server crash prevent
✔ ML inference overload বন্ধ
✔ Abuse control

🔥 3️⃣ Auto Add Custom Response Header

Production API তে versioning দরকার হয়।

```
class APIVersionMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        response = self.get_response(request)
        response["X-API-Version"] = "v1.0"
        return response
```

কেন?

✔ Frontend জানবে API version
✔ Future backward compatibility

🔥 4️⃣ Force HTTPS Redirect (Security)

Django এ production এ HTTPS বাধ্যতামূলক হওয়া উচিত।

```
from django.http import HttpResponsePermanentRedirect

class ForceHTTPSMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        if not request.is_secure():
            url = request.build_absolute_uri(request.get_full_path())
            secure_url = url.replace("http://", "https://")
            return HttpResponsePermanentRedirect(secure_url)

        return self.get_response(request)
```

কেন?

✔ Security
✔ SSL enforce
✔ SEO benefit

🔥 5️⃣ Country Block (Geo Restriction)

ধরো তুমি শুধু Bangladesh এ service দেবে।

```
class CountryBlockMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        country = request.META.get("GEOIP_COUNTRY_CODE")

        if country != "BD":
            return JsonResponse(
                {"error": "Service not available in your country"},
                status=403
            )

        return self.get_response(request)
```

কেন?

✔ Geo restriction
✔ Compliance
✔ Business control

🔥 6️⃣ Request ID Generator (Enterprise Level)

Large system এ প্রতিটা request এর unique ID দরকার হয় debugging এর জন্য।

```
import uuid

class RequestIDMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        request.id = str(uuid.uuid4())
        response = self.get_response(request)
        response["X-Request-ID"] = request.id
        return response
```

কেন?

✔ Log tracing
✔ Error tracking
✔ Microservice debugging

🧠 Real কোম্পানিগুলো middleware দিয়ে কী করে?

- Logging system
- Authentication token check
- Rate limiting
- A/B testing
- Feature flag control
- Analytics tracking
- Security filtering
- Performance monitoring

🎯 তোমার ML Surveillance Project এ Best Middleware Combo

আমি recommend করবো:

1. ✅ Request Logging
2. ✅ Inference Timing
3. ✅ Request Size Limit
4. ✅ IP Block
5. ✅ Request ID

এগুলো দিলে তোমার project **junior level থেকে production level** হয়ে যাবে 🚀

খুব ভালো প্রশ্ন 🔥
এটাই middleware পুরো বুঝার key point।

🧠 Middleware থেকে response কোথায় show হবে?

Middleware থেকে return করা response **সরাসরি browser / frontend এ পাঠানো হয়**।

মানে —

```
User Request
    ↓
Middleware (response return করলে এখানেই থেমে যাবে)
    ↓
View (যাবে না যদি middleware আগে return করে)
```

👉 যদি middleware এর ভিতরে `return JsonResponse(...)` দাও, তাহলে:

- View execute হবে না
- Direct client সেই response পাবে

🔥 Example 1 — Middleware থেকে direct response

```
from django.http import JsonResponse

class BlockMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        if request.path == "/blocked/":
```

```
            return JsonResponse(
                {"error": "Access denied"},
                status=403
            )

        return self.get_response(request)
```

🔎 এখন কী হবে?

User যদি `/blocked/` এ যায়:

✔ Middleware response return করবে
✔ View run হবে না
✔ Browser এ দেখাবে:

```
{
  "error": "Access denied"
}
```

⏰ Middleware কখন response পাঠায়?

Middleware দুইভাবে response পাঠাতে পারে:

🟢 Case 1: View এর আগে (Before View)

যখন middleware early return করে।

```
if condition:
    return response
```

⛔ View execute হবে না।

🟢 Case 2: View এর পরে (After View)

```
response = self.get_response(request)
response["X-Test"] = "Hello"
return response
```

✔ View আগে run হবে
✔ তারপর middleware response modify করবে
✔ তারপর user এ যাবে

🔄 Complete Flow বুঝো

ধরো তোমার ২টা middleware আছে:

```
MIDDLEWARE = [
    'core.middleware.FirstMiddleware',
    'core.middleware.SecondMiddleware',
]
```

Flow হবে:

```
Request
 ↓
FirstMiddleware (before)
 ↓
SecondMiddleware (before)
 ↓
View
 ↑
SecondMiddleware (after)
 ↑
FirstMiddleware (after)
 ↑
Response
```

👉 Request উপরের দিক দিয়ে যায়
👉 Response নিচের দিক দিয়ে ফিরে আসে

🎯 Real Example (Timing + Response)

```
import time

class TimingMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        start = time.time()
```

```
        response = self.get_response(request)

        duration = time.time() - start
        response["X-Response-Time"] = str(duration)

        return response
```

🔎 এটা কোথায় show করবে?

Browser DevTools → Network → Response Headers

```
X-Response-Time: 0.234
```

🧠 গুরুত্বপূর্ণ বুঝার বিষয়

Situation	View Run হবে?	Response কোথা থেকে যাবে?
Middleware early return	❌ না	Middleware
Middleware শুধু modify করে	✅ হ্যাঁ	View → Middleware
কোনো middleware নেই	✅ হ্যাঁ	View

🎯 তোমার ML API তে Example

ধরো file size বেশি:

Middleware:

```
return JsonResponse({"error": "Too large"}, status=413)
```

User সাথে সাথে 413 response পাবে।
Model run হবে না।
Server safe থাকবে। 🔥

💡 Short Answer

👉 Middleware response browser এ show করে
👉 দুই সময় return করতে পারে:

- View এর আগে
- View এর পরে

দারুণ 🔥 এখন আমরা শিখবো **Django middleware debugging কিভাবে করতে হয় (real practical way)** — যেটা production project এ খুব দরকার।

আমি step-by-step দেখাচ্ছি 👇

🧠 1️⃣ সবচেয়ে সহজ Debugging → `print()` ব্যবহার

ধরো তোমার middleware:

```
class TestMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        print("Before View")

        response = self.get_response(request)

        print("After View")

        return response
```

কীভাবে debug করবে?

Server run করো:

```
python manage.py runserver
```

Browser এ request পাঠাও।

👉 Terminal এ দেখবে:

```
Before View
After View
```

🧠 2️⃣ Check করো View run হচ্ছে কিনা

Middleware early return করলে view run হয় না।

Debug trick:

```
class DebugMiddleware:
    def __call__(self, request):
        print("Middleware start")

        if request.path == "/test/":
            print("Blocked here")
            return JsonResponse({"error": "blocked"})

        response = self.get_response(request)

        print("Middleware end")
        return response
```

এখন দেখবে:

✔ `/test/` এ গেলে → `"Middleware end"` print হবে না
✔ অন্য URL এ গেলে → সব print হবে

👉 বুঝতে পারবে কোথায় flow থেমেছে

🧠 3️⃣ Logging ব্যবহার (Production Way)

Django production এ `print()` ব্যবহার করা উচিত না।

settings.py এ logging add করো:

```
LOGGING = {
    "version": 1,
    "handlers": {
        "console": {
            "class": "logging.StreamHandler",
        },
    },
    "root": {
        "handlers": ["console"],
        "level": "INFO",
    },
}
```

Middleware এ:

```
import logging

logger = logging.getLogger(__name__)

class DebugMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        logger.info(f"Request path: {request.path}")

        response = self.get_response(request)

        logger.info(f"Response status: {response.status_code}")

        return response
```

✔ এখন সব clean ভাবে terminal এ দেখাবে
✔ Production safe

🧠 4️⃣ Middleware Order Debug করা

অনেক সময় problem হয় middleware order নিয়ে।

`settings.py`:

```
MIDDLEWARE = [
    'core.middleware.First',
    'core.middleware.Second',
]
```

Debug করতে:

```
print("First before")
print("Second before")
print("Second after")
print("First after")
```

👉 Flow ঠিক আছে কিনা বুঝবে

🧠 5️⃣ Exception Debug করা (Very Important)

Middleware এ error হলে server crash হতে পারে।

Example:

```
class ErrorDebugMiddleware:
    def __call__(self, request):
        try:
            response = self.get_response(request)
            return response
        except Exception as e:
            print("Middleware Error:", e)
            raise e
```

👉 এতে exact error দেখবে

🧠 6️⃣ Browser DevTools দিয়ে Debug

Middleware যদি response modify করে:

```
response["X-Test"] = "Hello"
```

Debug করতে:

Chrome → DevTools → Network → Headers

✔ Header এসেছে কিনা দেখো
✔ Status code check করো

🧠 7️⃣ Advanced Debug (Timing)

```
import time

class TimingDebugMiddleware:
    def __call__(self, request):
        start = time.time()
        response = self.get_response(request)
        duration = time.time() - start

        print(f"Time taken: {duration}")
```

```
        return response
```

👉 Slow API detect করতে পারবে

🔥 Middleware Debug Checklist

Problem	কী check করবে
View run হচ্ছে না	Early return আছে কিনা
Header আসছে না	Response modify হয়েছে কিনা
Error হচ্ছে	try-except দিয়ে ধরো
Order issue	settings.py order
Print দেখাচ্ছে না	Middleware add করা হয়েছে কিনা

🎯 তোমার ML Project এ Debug Strategy

যেহেতু তুমি ML inference server বানাচ্ছো:

✔ Model run হচ্ছে কিনা
✔ Inference time
✔ File size check
✔ User authentication

সব middleware দিয়ে log করলে debugging সহজ হবে।

হ্যাঁ ✅ **Device information check বা store করা middleware দিয়ে করা যায়** — এবং real-world এ এটা খুব common practice।

তুমি করতে পারো:

- 📱 User কোন device ব্যবহার করছে (Mobile / Desktop)
- 🌍 Browser কি
- 🖥️ OS কি
- 🌐 IP address
- 🔐 Suspicious device detect

এখন আমি practical ভাবে দেখাচ্ছি 👇

🧠 Device Information কিভাবে পাওয়া যায়?

সব information আসে `request.META` থেকে।

সবচেয়ে গুরুত্বপূর্ণ field:

```
request.META.get("HTTP_USER_AGENT")
```

এটা browser/device info দেয়।

Example user agent:

```
Mozilla/5.0 (Linux; Android 13; Pixel 6)
```

✅ Example 1 — শুধু check করা (Store না করে)

```
class DeviceCheckMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        user_agent = request.META.get("HTTP_USER_AGENT", "")

        if "Mobile" in user_agent:
            print("User is on Mobile")
        else:
            print("User is on Desktop")

        return self.get_response(request)
```

✔ শুধু detect করবে
✔ কিছু store করবে না

✅ Example 2 — Database এ Store করা (Production Way)

ধরো তুমি login এর সময় device info save করতে চাও।

Model বানাও:

```python
# models.py

from django.db import models
from django.contrib.auth.models import User

class DeviceLog(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE, null=True)
    ip_address = models.GenericIPAddressField()
    user_agent = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)
```

Middleware:

```python
from .models import DeviceLog

class DeviceStoreMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        response = self.get_response(request)

        if request.user.is_authenticated:
            DeviceLog.objects.create(
                user=request.user,
                ip_address=request.META.get("REMOTE_ADDR"),
                user_agent=request.META.get("HTTP_USER_AGENT")
            )

        return response
```

🔥 কখন middleware দিয়ে এটা করা উচিত?

✔ Login tracking
✔ Suspicious activity detect
✔ Multiple device login monitor
✔ Security audit
✔ Admin panel analytics

🚨 Important Warning

Middleware এ **প্রতিটা request এ database write করা উচিত না** ❌
কারণ:

- Performance slow হবে
- DB overload হবে

Better practice:

✔ শুধু login request এ log করা
✔ শুধু specific endpoint এ log করা
✔ Redis cache ব্যবহার করা

🔥 Advanced Way (Better Device Detection)

Raw user agent parsing কঠিন।

তুমি ব্যবহার করতে পারো:

Django project এ third-party package:

```
pip install django-user-agents
```

এটা automatically detect করবে:

- is_mobile
- is_tablet
- is_pc
- browser family
- OS family

🎯 তোমার ML Surveillance Project এ Use Case

তুমি করতে পারো:

- কে কোন device দিয়ে video upload করছে
- Suspicious IP + suspicious device combo detect
- Multiple login from different device alert

💡 Final Answer

👉 হ্যাঁ, middleware দিয়ে device info check ও store করা যায়
👉 কিন্তু সব request এ DB write করা উচিত না
👉 Smart filtering করতে হবে

চাও কি আমি দেখাই:

- 🔥 Device change detect করলে alert কিভাবে দিবে
- 🔥 Login location + device mismatch detect
- 🔥 Middleware vs Signal (কোনটা better এই কাজের জন্য)

কোনটা জানতে চাও? 🚀
দারুণ 🔥 এখন আমরা production-level security logic শিখবো — যেটা বড় কোম্পানিগুলো ব্যবহার করে।

আমি ৩টা অংশে explain করছি:

1️⃣ Device change detect + alert
2️⃣ Login location + device mismatch detect
3️⃣ Middleware vs Signal — কোনটা better

✅ 1️⃣ Device Change Detect করলে Alert কিভাবে দিবে?

🎯 Idea

User login করলে:

- আগের device info DB থেকে আনবে
- নতুন device info compare করবে
- আলাদা হলে → alert দিবে

Step 1: Model

```
# models.py

from django.db import models
from django.contrib.auth.models import User

class UserDevice(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    device_info = models.TextField()
```

```
    ip_address = models.GenericIPAddressField()
    last_login = models.DateTimeField(auto_now=True)
```

Step 2: Middleware (Login Endpoint only)

```
from .models import UserDevice
from django.core.mail import send_mail

class DeviceAlertMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        response = self.get_response(request)

        if request.path == "/login/" and request.user.is_authenticated:
            user_agent = request.META.get("HTTP_USER_AGENT")
            ip = request.META.get("REMOTE_ADDR")

            last_device =
UserDevice.objects.filter(user=request.user).first()

            if last_device:
                if last_device.device_info != user_agent:
                    # Alert
                    send_mail(
                        "New Device Login Alert",
                        "Your account was accessed from a new device.",
                        "noreply@example.com",
                        [request.user.email],
                    )

            UserDevice.objects.update_or_create(
                user=request.user,
                defaults={
                    "device_info": user_agent,
                    "ip_address": ip
                }
            )

        return response
```

🔥 কী হবে?

✔ নতুন device হলে email alert যাবে
✔ পুরোনো হলে কিছু হবে না

✅ 2️⃣ Login Location + Device Mismatch Detect

এটা enterprise security 🔥

🎯 Logic:

- আগের IP country
- নতুন IP country
- যদি country change + device change → high risk

Country detect করা যায়:

IP → GeoIP database ব্যবহার করে।

Django এ built-in GeoIP support আছে:

Django

Example Logic:

```
if last_device:
    if last_device.ip_address != ip:
        print("IP changed")

    if last_device.device_info != user_agent:
        print("Device changed")

    if both_changed:
        print("High risk login detected")
```

🚨 Real System কী করে?

Condition	Action
Device change	Email alert

Condition	Action
Country change	Email + OTP verify
Both change	Force re-login + OTP
Suspicious country	Block

✅ 3️⃣ Middleware vs Signal — কোনটা better?

এটা খুব important 🔥

🧠 Middleware

✔ সব request এ কাজ করে
✔ Global control
✔ Request/Response access আছে

❌ Login specific logic এর জন্য heavy

🧠 Django Signal (Better for Login Tracking)

Django built-in signal আছে:

```
from django.contrib.auth.signals import user_logged_in
```

Signal শুধু login হলে trigger হবে।

Example (Better Way)

```
from django.contrib.auth.signals import user_logged_in
from django.dispatch import receiver

@receiver(user_logged_in)
def detect_device_change(sender, request, user, **kwargs):
    user_agent = request.META.get("HTTP_USER_AGENT")
    ip = request.META.get("REMOTE_ADDR")

    # compare + alert logic here
```

🏆 Final Verdict

কাজ	Best Option
Login tracking	✅ Signal
All API monitoring	✅ Middleware
Security filtering	✅ Middleware
Device alert	✅ Signal
Request modification	✅ Middleware

🎯 তোমার ML Surveillance Project এ কী করবো?

আমি recommend করবো:

✔ Login security → Signal
✔ API abuse detect → Middleware
✔ Inference tracking → Middleware
✔ Suspicious login alert → Signal

🔥 Production Level Security Flow

Login
⬇
Signal trigger
⬇
Device compare
⬇
IP compare
⬇
Risk score calculate
⬇
Email / OTP / Block

চাও কি আমি এখন দেখাই:

- 🔥 Risk score system (Google style security)
- 🔥 OTP verify on suspicious login

- 🔥 Redis use করে device cache করা
- 🔥 Complete secure login architecture diagram

কোনটা next? 🚀
দারুণ 🔥
তুমি চাও **middleware + login security system পুরো first থেকে last পর্যন্ত শিখতে**, যেন যেকোনো project এ ব্যবহার করতে পারো।

আমি এখন তোমাকে **একটা complete, reusable architecture** শেখাবো — যেটা যেকোনো Django project এ use করতে পারবে।

🎯 আমরা কী বানাবো?

একটা secure login monitoring system যেখানে থাকবে:

1️⃣ Device detect
2️⃣ IP detect
3️⃣ Location detect (basic level)
4️⃣ Device change detect
5️⃣ Risk check
6️⃣ Email alert
7️⃣ Reusable structure

🏗️ Step 1 — Project Structure

```
project/
 ├── core/
 │    ├── middleware.py
 │    ├── signals.py
 │    ├── models.py
 │    └── utils.py
```

🧠 Step 2 — Device Log Model (Reusable)

```
# core/models.py

from django.db import models
from django.contrib.auth.models import User

class LoginActivity(models.Model):
```

```
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    ip_address = models.GenericIPAddressField()
    user_agent = models.TextField()
    device_type = models.CharField(max_length=50)
    country = models.CharField(max_length=100, null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.user} - {self.device_type}"
```

✔ যেকোনো project এ use করা যাবে
✔ Extend করা যাবে

🧠 Step 3 — Device Detect Utility

```
# core/utils.py

def get_device_type(user_agent):
    if "Mobile" in user_agent:
        return "Mobile"
    elif "Tablet" in user_agent:
        return "Tablet"
    else:
        return "Desktop"
```

🧠 Step 4 — Login Signal (Best Practice)

Login detect করার জন্য signal ব্যবহার করবো (middleware না)

```
# core/signals.py

from django.contrib.auth.signals import user_logged_in
from django.dispatch import receiver
from .models import LoginActivity
from .utils import get_device_type
from django.core.mail import send_mail

@receiver(user_logged_in)
def log_login_activity(sender, request, user, **kwargs):
    ip = request.META.get("REMOTE_ADDR")
    user_agent = request.META.get("HTTP_USER_AGENT", "")
    device_type = get_device_type(user_agent)
```

```
    last_login = LoginActivity.objects.filter(user=user).order_by("-created_at").first()

    risk = False

    if last_login:
        if last_login.device_type != device_type:
            risk = True

        if last_login.ip_address != ip:
            risk = True

    # Save login activity
    LoginActivity.objects.create(
        user=user,
        ip_address=ip,
        user_agent=user_agent,
        device_type=device_type,
    )

    # Send alert if risk detected
    if risk:
        send_mail(
            "Suspicious Login Detected",
            "We detected login from new device or IP.",
            "noreply@example.com",
            [user.email],
        )
```

🧠 Step 5 — AppConfig এ signal connect করো

```
# core/apps.py

from django.apps import AppConfig

class CoreConfig(AppConfig):
    name = "core"

    def ready(self):
        import core.signals
```

🧠 Step 6 — Middleware কোথায় ব্যবহার করবো?

Middleware ব্যবহার করবো:

✔ Request logging
✔ Rate limiting
✔ Request size control
✔ Security headers

Example:

```
# core/middleware.py

import time

class RequestTimingMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        start = time.time()
        response = self.get_response(request)
        response["X-Response-Time"] = str(time.time() - start)
        return response
```

🧠 Step 7 — Risk Level System (Reusable Pattern)

```
def calculate_risk(last_login, current_ip, current_device):
    score = 0

    if last_login.ip_address != current_ip:
        score += 50

    if last_login.device_type != current_device:
        score += 50

    return score
```

Score	Action
0	Safe
50	Email alert
100	Force OTP

🧠 Middleware vs Signal — Final Rule (Remember This)

কাজ	Use
Login specific event	✅ Signal
Global request control	✅ Middleware
Security headers	✅ Middleware
Suspicious login alert	✅ Signal
Performance tracking	✅ Middleware

🏆 এখন তুমি কী শিখলে?

✔ Device detect
✔ IP detect
✔ Risk calculate
✔ Email alert
✔ Reusable structure
✔ Middleware কোথায় লাগবে
✔ Signal কোথায় লাগবে

🚀 এই system যেকোনো project এ ব্যবহার করা যাবে:

- E-commerce
- SaaS
- ML API
- Admin dashboard
- Banking type system
- School management

🔥 Next Level (Professional)

চাও কি আমি শেখাই:

- 🔐 Suspicious login হলে OTP verify system
- 🔥 Redis use করে login cache করা

- 🌍 Country detection full setup
- 📊 Admin dashboard এ login analytics
- 🏢 Enterprise level authentication architecture

কোনটা next শিখতে চাও?

দারুণ 🔥 এখন আমরা একটা **enterprise-style secure authentication system** শিখবো — যেটা যেকোনো Django project এ reuse করতে পারবে।

আমরা ৫টা অংশে পুরো architecture বানাবো:

1️⃣ Suspicious login → OTP verification
2️⃣ Redis দিয়ে login cache
3️⃣ Country detection setup
4️⃣ Admin dashboard analytics
5️⃣ Enterprise authentication architecture

🔐 1️⃣ Suspicious Login হলে OTP Verify System

🎯 Flow

Login
⬇
Signal trigger
⬇
Risk score calculate
⬇
High risk হলে → OTP generate
⬇
User OTP verify না করলে full access পাবে না

Step 1 — OTP Model

```
# core/models.py

class LoginOTP(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    otp = models.CharField(max_length=6)
    created_at = models.DateTimeField(auto_now_add=True)
    is_verified = models.BooleanField(default=False)
```

Step 2 — OTP Generate Function

```
import random

def generate_otp():
    return str(random.randint(100000, 999999))
```

Step 3 — Suspicious Login Signal

```
@receiver(user_logged_in)
def handle_login(sender, request, user, **kwargs):
    ip = request.META.get("REMOTE_ADDR")
    user_agent = request.META.get("HTTP_USER_AGENT", "")

    last = LoginActivity.objects.filter(user=user).order_by("-
created_at").first()

    risk_score = 0
    if last:
        if last.ip_address != ip:
            risk_score += 50
        if last.user_agent != user_agent:
            risk_score += 50

    if risk_score >= 100:
        otp = generate_otp()

        LoginOTP.objects.create(user=user, otp=otp)

        send_mail(
            "OTP Verification Required",
            f"Your OTP is {otp}",
            "noreply@example.com",
            [user.email],
        )
```

Step 4 — OTP Verify API

```
def verify_otp(request):
    otp_input = request.POST.get("otp")

    record = LoginOTP.objects.filter(
```

```
        user=request.user,
        otp=otp_input,
        is_verified=False
    ).first()

    if record:
        record.is_verified = True
        record.save()
        return JsonResponse({"message": "Verified"})

    return JsonResponse({"error": "Invalid OTP"}, status=400)
```

🔥 2️⃣ Redis ব্যবহার করে Login Cache করা

DB write slow হয়। তাই Redis ব্যবহার করবো।

Install:

```
pip install django-redis
```

---

settings.py

```
CACHES = {
    "default": {
        "BACKEND": "django_redis.cache.RedisCache",
        "LOCATION": "redis://127.0.0.1:6379/1",
    }
}
```

Use Redis for temporary OTP

```
from django.core.cache import cache

def store_otp(user_id, otp):
    cache.set(f"otp_{user_id}", otp, timeout=300)  # 5 min

def get_otp(user_id):
    return cache.get(f"otp_{user_id}")
```

✔ Fast
✔ No DB load
✔ Auto expire

🌍 3️⃣ Country Detection Full Setup

Django built-in GeoIP আছে:

Django

Install GeoIP database

```
pip install geoip2
```

Download MaxMind GeoLite2 database।

---

settings.py

```
GEOIP_PATH = BASE_DIR / "geoip"
```

Use in signal:

```
from django.contrib.gis.geoip2 import GeoIP2

g = GeoIP2()
country = g.country(ip)["country_name"]
```

Risk Logic

Change	Score
Device change	+50
IP change	+50
Country change	+100

Country change হলে:

✔ Force OTP
✔ Send email alert

📊 4️⃣ Admin Dashboard Login Analytics

Model already আছে: `LoginActivity`

Now Django admin এ register করো:

```
# admin.py

from django.contrib import admin
from .models import LoginActivity

@admin.register(LoginActivity)
class LoginActivityAdmin(admin.ModelAdmin):
    list_display = ("user", "ip_address", "device_type", "country",
"created_at")
    list_filter = ("device_type", "country")
```

Admin এ দেখতে পারবে:

✔ কে কখন login করেছে
✔ কোন country থেকে
✔ কোন device
✔ Suspicious pattern

🏢 5️⃣ Enterprise Level Authentication Architecture

🔥 Full Security Flow

```
User Login
    ↓
Authentication
    ↓
Signal Trigger
    ↓
Device/IP/Country Compare
    ↓
```

```
Risk Score Calculate
    ↓
Low Risk → Normal Login
Medium Risk → Email Alert
High Risk → OTP Verification
Very High Risk → Block Account
```

🏆 Enterprise Best Practices

✔ Redis for temporary data
✔ DB only for permanent logs
✔ Risk scoring system
✔ GeoIP detection
✔ OTP expiry
✔ Suspicious login dashboard
✔ Background email queue (Celery recommended)

🧠 Middleware কোথায় ব্যবহার হবে এখানে?

✔ Rate limiting
✔ API monitoring
✔ Security headers
✔ Suspicious IP block

Signal ব্যবহার হবে:

✔ Login event detect
✔ OTP trigger

🎯 এখন তুমি পারবে

- Banking style login system বানাতে
- SaaS authentication system বানাতে
- ML API secure করতে
- Multi-device login detect করতে
- Enterprise level security flow implement করতে

🚀 Next Level (Very Advanced)

চাও কি আমি শেখাই:

- 🔥 JWT + Device binding system
- 🔥 Account lock after multiple failed login
- 🔥 Two-factor authentication (Google Authenticator style)
- 🔥 Full microservice authentication architecture
- 🔥 Production deployment security checklist

কোনটা next শিখতে চাও?
দারুণ 🔥 তুমি চাইছো **Production Deployment এর সময় কী কী security check করতে হবে**।
আমি একদম detail দিয়ে step-by-step guide দিচ্ছি, যা **যেকোনো Django + ML API project** এ use করা যাবে।

🏗️ Django / ML API Production Deployment Security Checklist

1️⃣ Server & OS Security

- ✅ Use latest OS version and security patches
- ✅ Disable root login via SSH
- ✅ Use strong passwords & SSH key authentication
- ✅ Firewall setup (ufw/iptables)
- ✅ Fail2ban বা অন্যান্য brute-force protection
- ✅ Disable unused services / ports

2️⃣ HTTPS / SSL

- ✅ Use **HTTPS** for all endpoints (Let's Encrypt / Paid SSL)
- ✅ Redirect all HTTP → HTTPS
- ✅ HSTS header set (HTTP Strict Transport Security)

```
SECURE_HSTS_SECONDS = 3600
SECURE_HSTS_INCLUDE_SUBDOMAINS = True
SECURE_HSTS_PRELOAD = True
```

- ✅ Secure cookies (`SESSION_COOKIE_SECURE` , `CSRF_COOKIE_SECURE`)

3️⃣ Django Security Settings

```
DEBUG = False
ALLOWED_HOSTS = ['yourdomain.com']
SECRET_KEY = env_secret_key
CSRF_COOKIE_SECURE = True
SESSION_COOKIE_SECURE = True
SECURE_BROWSER_XSS_FILTER = True
SECURE_CONTENT_TYPE_NOSNIFF = True
X_FRAME_OPTIONS = 'DENY'
```

- ✅ Use environment variables for **SECRET_KEY**
- ✅ `DEBUG=False` in production

4️⃣ Database Security

- ✅ Use strong DB password
- ✅ Only allow DB access from server (not public)
- ✅ Database user limited permissions (no root)
- ✅ Use SSL connection if DB is remote

5️⃣ Authentication & Authorization

- ✅ Enforce strong password policy
- ✅ Multi-device login detection
- ✅ Two-factor authentication (2FA / OTP)
- ✅ Limit failed login attempts (lockout after 5-10 attempts)
- ✅ JWT token / session expiry
- ✅ Role-based access control for admin APIs

6️⃣ Middleware & Request Security

- ✅ Rate limiting to prevent brute-force / DDoS
- ✅ Request size limiting (especially for ML file uploads)
- ✅ IP block suspicious users
- ✅ Logging of all API requests (device, IP, endpoint, time)
- ✅ Security headers

```
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-XSS-Protection: 1; mode=block
```

7️⃣ ML Model / API Security

- ✅ Validate uploaded files (extension, MIME type, size)
- ✅ Scan files for malicious content
- ✅ Sandbox / isolated process for inference
- ✅ Rate limit heavy ML endpoints
- ✅ Monitor inference time (slow requests alert)

8️⃣ Caching / Performance Security

- ✅ Use Redis / Memcached for temporary storage (OTP, tokens)
- ✅ Limit cache exposure to internal network
- ✅ Use TTL for sensitive data

9️⃣ Logging & Monitoring

- ✅ Centralized logging (ELK / Graylog / Papertrail)
- ✅ Track suspicious logins, failed logins, and device changes
- ✅ Monitor server CPU, memory, disk, network
- ✅ Alert system for high-risk events (email / Slack / SMS)
- ✅ Audit logs for admin actions

🔟 Backup & Disaster Recovery

- ✅ Daily DB backup & encrypted storage
- ✅ Backup rotation policy
- ✅ Test restore process
- ✅ Store ML model versions safely

1️⃣1️⃣ Dependency & Code Security

- ✅ Pin package versions (`requirements.txt`)
- ✅ Use virtualenv / pipenv / poetry
- ✅ Scan dependencies for vulnerabilities (`safety` , `pip-audit`)
- ✅ Disable debug logs in production

1️⃣2️⃣ Container / Deployment Security (Optional)

- ✅ Use Docker / Kubernetes for isolation
- ✅ Limit container privileges
- ✅ Read-only filesystem if possible
- ✅ Health checks & resource limits

1️⃣3️⃣ Extra Security Layers

- ✅ Captcha on login / signup
- ✅ Email verification on signup / new device login
- ✅ GeoIP detection → Block high-risk countries
- ✅ Device fingerprinting → Alert new device login

🔥 Production Ready Flow for Your ML Project

```
User → HTTPS → Firewall → Django Middleware → DRF API → ML Model →
Response
                      ↓
                 Logging & Monitoring
                      ↓
            Suspicious Login Detection
                      ↓
                 OTP / Alerts
```

🧠 TL;DR

1. OS + server secure
2. HTTPS + secure cookies
3. Debug OFF, ALLOWED_HOSTS set
4. Strong auth + 2FA

5. Request validation + rate limiting
6. Log + monitor suspicious activity
7. Backup + disaster recovery
8. Dependency & container security
9. Admin dashboard analytics for risk detection

💡 যদি চাও, আমি তোমার জন্য **একটা ready-to-use "Production Deployment Checklist + ML API Secure Template" বানিয়ে দিতে পারি**, যেখানে সব setting, middleware, Redis, OTP, GeoIP, logging included থাকবে — শুধু copy paste করলেই হবে।

চাও আমি সেটা বানাই?

চমৎকার! 🚀 আমি এখন তোমার জন্য **complete ready-to-use Production Deployment + ML API Security Template** বানিয়ে দিচ্ছি, যাতে সবকিছু included থাকবে:

- Middleware
- Redis cache
- OTP verification
- GeoIP detection
- Logging
- ML API safe handling
- Production-ready settings

তোমার জন্য structure এমন হবে যে **copy-paste করলেই project ready হবে**।

📦 Project Structure

```
project/
 ├── core/
 │    ├── middleware.py
 │    ├── signals.py
 │    ├── models.py
 │    ├── utils.py
 │    └── apps.py
 ├── project/
 │    ├── settings.py
 │    ├── urls.py
 │    └── wsgi.py
 ├── requirements.txt
 └── manage.py
```

1️⃣ requirements.txt

```
Django>=4.2
djangorestframework
django-redis
geoip2
```

2️⃣ core/models.py

```python
from django.db import models
from django.contrib.auth.models import User

class LoginActivity(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    ip_address = models.GenericIPAddressField()
    user_agent = models.TextField()
    device_type = models.CharField(max_length=50)
    country = models.CharField(max_length=100, null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

class LoginOTP(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    otp = models.CharField(max_length=6)
    created_at = models.DateTimeField(auto_now_add=True)
    is_verified = models.BooleanField(default=False)
```

3️⃣ core/utils.py

```python
import random
from django.core.cache import cache
from django.contrib.gis.geoip2 import GeoIP2

def get_device_type(user_agent):
    if "Mobile" in user_agent:
        return "Mobile"
    elif "Tablet" in user_agent:
        return "Tablet"
    return "Desktop"

def generate_otp():
    return str(random.randint(100000, 999999))
```

```
def store_otp(user_id, otp, timeout=300):
    cache.set(f"otp_{user_id}", otp, timeout=timeout)

def get_otp(user_id):
    return cache.get(f"otp_{user_id}")

def get_country(ip):
    try:
        g = GeoIP2()
        return g.country(ip)["country_name"]
    except:
        return "Unknown"
```

4️⃣ core/signals.py

```
from django.contrib.auth.signals import user_logged_in
from django.dispatch import receiver
from django.core.mail import send_mail
from .models import LoginActivity, LoginOTP
from .utils import get_device_type, generate_otp, store_otp, get_country

@receiver(user_logged_in)
def log_login_activity(sender, request, user, **kwargs):
    ip = request.META.get("REMOTE_ADDR")
    user_agent = request.META.get("HTTP_USER_AGENT", "")
    device_type = get_device_type(user_agent)
    country = get_country(ip)

    last_login = LoginActivity.objects.filter(user=user).order_by("-
created_at").first()
    risk_score = 0

    if last_login:
        if last_login.device_type != device_type:
            risk_score += 50
        if last_login.ip_address != ip:
            risk_score += 50
        if last_login.country != country:
            risk_score += 100

    LoginActivity.objects.create(
        user=user,
        ip_address=ip,
        user_agent=user_agent,
        device_type=device_type,
        country=country
```

```
    )

    if risk_score >= 100:
        otp = generate_otp()
        store_otp(user.id, otp)
        send_mail(
            "Suspicious Login OTP",
            f"Your OTP is: {otp}",
            "noreply@example.com",
            [user.email]
        )
```

5️⃣ core/middleware.py

```
import time
from django.http import JsonResponse
from django.core.cache import cache

class RequestTimingMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        start = time.time()

        # Optional: Rate Limiting Example
        ip = request.META.get("REMOTE_ADDR")
        count = cache.get(f"rl_{ip}", 0)
        if count > 50:
            return JsonResponse({"error": "Too many requests"},
status=429)
        cache.set(f"rl_{ip}", count+1, timeout=60)  # 1 min

        response = self.get_response(request)
        response["X-Response-Time"] = str(time.time() - start)
        return response

class SecureHeadersMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        response = self.get_response(request)
        response["X-Frame-Options"] = "DENY"
        response["X-Content-Type-Options"] = "nosniff"
```

```
        response["X-XSS-Protection"] = "1; mode=block"
        return response
```

6️⃣ core/apps.py

```
from django.apps import AppConfig

class CoreConfig(AppConfig):
    default_auto_field = "django.db.models.BigAutoField"
    name = "core"

    def ready(self):
        import core.signals
```

7️⃣ project/settings.py (Production Ready Security)

```
DEBUG = False
ALLOWED_HOSTS = ['yourdomain.com']

SECRET_KEY = 'your-secret-key'

# HTTPS & Cookies
SECURE_SSL_REDIRECT = True
CSRF_COOKIE_SECURE = True
SESSION_COOKIE_SECURE = True
SECURE_HSTS_SECONDS = 3600
SECURE_HSTS_INCLUDE_SUBDOMAINS = True
SECURE_HSTS_PRELOAD = True

# Middleware
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'core.middleware.RequestTimingMiddleware',
    'core.middleware.SecureHeadersMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
]

# Redis cache
CACHES = {
```

```
    "default": {
        "BACKEND": "django_redis.cache.RedisCache",
        "LOCATION": "redis://127.0.0.1:6379/1",
        "OPTIONS": {"CLIENT_CLASS": "django_redis.client.DefaultClient"},
    }
}

# Logging
LOGGING = {
    "version": 1,
    "handlers": {"console": {"class": "logging.StreamHandler"}},
    "root": {"handlers": ["console"], "level": "INFO"},
}
```

8️⃣ ML API Safe Handling Example

```
from rest_framework.decorators import api_view
from rest_framework.response import Response

@api_view(["POST"])
def ml_predict(request):
    file = request.FILES.get("video")
    if not file:
        return Response({"error": "File required"}, status=400)
    if file.size > 50*1024*1024:  # 50MB limit
        return Response({"error": "File too large"}, status=413)

    # TODO: Run ML model safely
    result = {"prediction": "safe"}
    return Response(result)
```

✅ Key Features Included

- Middleware: Request timing, rate limiting, security headers
- Redis cache: OTP & rate limiting
- OTP verification for suspicious login
- GeoIP country detection
- Device & IP logging
- Production-ready Django settings
- ML API file validation
- Logging & monitoring

💡 **Next Step:**

তুমি চাইলে আমি এই template এর সাথে **OTP verify API + Admin Dashboard + GeoIP map + Enterprise Risk Score system** integrate করে **full ready-to-deploy secure ML API** বানিয়ে দিতে পারি।

চাও আমি সেটা বানাই?
ঠিক আছে, চল summary দেই — আজ তুমি **প্রায় সব security + middleware + login monitoring + production deployment** বিষয় একদম end-to-end শিখলে। 🔥

🧠 আজকের শেখার Summary

1️⃣ Middleware Basics

- Middleware হলো **request ও response এর মধ্যে intercept করার layer**
- কাজ:
  - Request modify করা
  - Response modify করা
  - Early return (view skip)
- Response কোথায় show হয়:
  - **View এর আগে** → direct browser
  - **View এর পরে** → response modify হয়ে browser

2️⃣ Middleware Practical Uses

- Device detection
- IP logging
- Rate limiting
- Request timing
- Security headers
- Suspicious API blocking

3️⃣ Debugging Middleware

- Print / Logging
- Check before/after view flow
- Track early return

- Use `logging` in production (avoid print)
- Debug request headers & response
- Measure request timing

4️⃣ Device & Login Security

- Device detect: `request.META["HTTP_USER_AGENT"]`
- IP detect: `request.META["REMOTE_ADDR"]`
- Device change detection → email alert
- Risk scoring: Device + IP + Country
- OTP verification for high-risk logins

5️⃣ Middleware vs Signal

কাজ	Best Use
Login-specific events	✅ Signal
Global request control	✅ Middleware
Suspicious login alerts	✅ Signal
Security headers / API monitoring	✅ Middleware

6️⃣ Production Deployment Security Checklist

- Server & OS hardening
- HTTPS + secure cookies + HSTS
- `DEBUG=False`, strong `SECRET_KEY`
- DB security + encrypted backup
- Middleware: Rate limiting, security headers, request validation
- Signal: Login monitoring, OTP alerts
- Redis: OTP caching, rate limiting
- Logging & monitoring (ELK / centralized)
- ML API security: file size check, validation, sandbox
- Admin dashboard: Login analytics & device tracking

7️⃣ Ready-to-Use Template Features

- Middleware: Timing, security headers, rate limiting
- Redis caching: OTP, rate limit, fast temp data
- Signal: Suspicious login detection & OTP
- GeoIP country detection
- ML API safe handling
- Logging of all login activity
- Production-ready Django settings

🔥 Outcome:

আজকে তুমি শিখলে:

1. **Middleware full workflow + debugging**
2. **Device/IP/Country login tracking**
3. **Suspicious login OTP system**
4. **Redis cache integration**
5. **Production-ready security checklist for Django + ML API**
6. **Reusable template ready for copy-paste**

💡 পরের step: আমরা চাইলে **full enterprise-ready secure ML API** বানাতে পারি, যেখানে:

- OTP verify + Redis
- Device/IP/Country risk score
- Admin analytics dashboard + GeoIP map
- Multi-device login handling

যদি চাও, আমি একদম **copy-paste deployable secure ML API full template** বানিয়ে দিতে পারি, যা production-ready।

চাও আমি বানাই?

হ্যাঁ, তুমি যদি **Django middleware expert level** এ যেতে চাও, তাহলে শুধু basic এবং production-ready ব্যবহার জানা যথেষ্ট নয়। কিছু extra knowledge থাকলে তুমি **অত্যন্ত flexible ও powerful middleware বানাতে পারবে যেকোনো project এ**।

আমি categorize করে explain করছি 👇

1️⃣ Middleware Lifecycle & Structure

- Understand `__init__`, `__call__`, `process_view`, `process_exception`, `process_template_response`
  - `__call__` → main method, every request pass
  - `process_view` → view execution আগে run হয়
  - `process_exception` → view বা middleware exception catch করে
  - `process_template_response` → TemplateResponse modify করার জন্য

> Expert tip: অনেক time-sensitive logging, caching, or performance measurement এর জন্য এই methods আলাদা ভাবে use করা হয়।

2️⃣ Middleware Order

- Middleware **execution order critical**
- Request → top-down execution
- Response → bottom-up execution
- Misordering → security / logging / session issues

> Example: `SecurityMiddleware` আগে, `AuthenticationMiddleware` পরে।

3️⃣ Performance & Scalability

- Middleware impact on **request/response latency**
- Avoid heavy computation in every request → use signals or async tasks
- Cache expensive operations (Redis, Memcached)

4️⃣ Conditional Middleware

- সব request এ middleware run করা উচিত নয়
- Use **custom logic** to skip certain paths or users

```
if request.path.startswith("/static/"):
    return self.get_response(request)
```

- Useful for ML API → skip file-heavy uploads for some middleware

---

5️⃣ Exception Handling in Middleware

- Catch exceptions to avoid server crash
- Log errors with traceback
- Optionally return custom error response

```
try:
    response = self.get_response(request)
except Exception as e:
    logger.error(f"Middleware error: {e}")
    return JsonResponse({"error": "Server error"}, status=500)
```

6️⃣ Async Middleware (Django 4+)

- Modern Django supports async middleware
- Important for **high throughput APIs** like ML inference server

```
class AsyncMiddleware:
    async def __call__(self, request):
        # async pre-processing
        response = await self.get_response(request)
        # async post-processing
        return response
```

7️⃣ Middleware + Security

- Add headers: `CSP`, `HSTS`, `X-Frame-Options`
- Rate limiting / brute force protection
- JWT validation / token refresh
- IP blocking / geolocation filtering

8️⃣ Middleware + Logging / Monitoring

- Request timing
- Device + IP logging
- Error / exception logging
- API usage analytics

> Combine with centralized logging tools: ELK, Sentry

9️⃣ Reusability & Modularity

- Write **configurable middleware** → use in multiple projects
- Use `settings.py` flags to enable/disable middleware
- Example:

```
if not getattr(settings, "ENABLE_SECURITY_MIDDLEWARE", True):
    return self.get_response(request)
```

1️⃣0️⃣ Testing Middleware

- Unit test middleware behavior
- Test:
  - Request modification
  - Response modification
  - Exception handling
  - Conditional skipping

> `RequestFactory` in Django helps test middleware easily

✅ TL;DR – Expert Level Middleware Needs

- Middleware lifecycle & methods
- Order & execution flow
- Async vs sync middleware
- Conditional execution
- Performance impact & caching
- Exception handling
- Security & logging integration
- Reusability & configuration
- Testing strategy

💡 **Next Step Suggestion:**

যদি চাও, আমি **একটা "Advanced Middleware Master Template" বানিয়ে দিতে পারি** যেখানে থাকবে:

- Async support
- Conditional paths
- Device/IP logging + Redis cache
- OTP + Suspicious login detection
- Full error handling + logging
- Production-ready security headers

> Copy-paste করলেই যেকোনো Django project এ use করা যাবে।

চাও আমি সেটা বানাই?

দারুণ! 🚀 তাহলে আমি তোমার জন্য **"Advanced Middleware Master Template"** বানিয়ে দিচ্ছি।

এই template **production-ready**, **async-ready**, এবং **reusable**। Features:

- Async support
- Conditional paths (skip middleware for certain URLs)
- Device/IP logging + Redis cache
- OTP + Suspicious login detection
- Full error handling + logging
- Security headers
- Copy-paste ready

📦 Project Structure

```
project/
 ├── core/
 │    ├── middleware.py
 │    ├── signals.py
 │    ├── models.py
 │    ├── utils.py
 │    └── apps.py
 ├── project/
 │    ├── settings.py
 │    ├── urls.py
 │    └── wsgi.py
 ├── requirements.txt
 └── manage.py
```

1️⃣ requirements.txt

```
Django>=4.2
djangorestframework
django-redis
geoip2
```

2️⃣ core/models.py

```python
from django.db import models
from django.contrib.auth.models import User

class LoginActivity(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    ip_address = models.GenericIPAddressField()
    user_agent = models.TextField()
    device_type = models.CharField(max_length=50)
    country = models.CharField(max_length=100, null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

class LoginOTP(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    otp = models.CharField(max_length=6)
    created_at = models.DateTimeField(auto_now_add=True)
    is_verified = models.BooleanField(default=False)
```

3️⃣ core/utils.py

```python
import random
from django.core.cache import cache
from django.contrib.gis.geoip2 import GeoIP2

def get_device_type(user_agent):
    if "Mobile" in user_agent:
        return "Mobile"
    elif "Tablet" in user_agent:
        return "Tablet"
    return "Desktop"

def generate_otp():
    return str(random.randint(100000, 999999))
```

```
def store_otp(user_id, otp, timeout=300):
    cache.set(f"otp_{user_id}", otp, timeout=timeout)

def get_otp(user_id):
    return cache.get(f"otp_{user_id}")

def get_country(ip):
    try:
        g = GeoIP2()
        return g.country(ip)["country_name"]
    except:
        return "Unknown"
```

4️⃣ core/middleware.py (Advanced, Async-ready)

```
import time
import logging
from django.http import JsonResponse
from django.core.cache import cache
from django.conf import settings
from .utils import get_device_type, store_otp, generate_otp, get_country
from .models import LoginActivity

logger = logging.getLogger(__name__)

class AdvancedSecurityMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    async def __call__(self, request):
        start_time = time.time()

        # ----- Conditional Path Skip -----
        skip_paths = getattr(settings, "MIDDLEWARE_SKIP_PATHS",
["/static/", "/media/"])
        if any(request.path.startswith(p) for p in skip_paths):
            return await self._get_response(request)

        try:
            response = await self._get_response(request)

            # ----- Device & IP Logging -----
            user = getattr(request, "user", None)
            if user and user.is_authenticated:
                ip = request.META.get("REMOTE_ADDR")
                user_agent = request.META.get("HTTP_USER_AGENT", "")
```

```
                device_type = get_device_type(user_agent)
                country = get_country(ip)

                last_login =
LoginActivity.objects.filter(user=user).order_by("-created_at").first()
                risk_score = 0
                if last_login:
                    if last_login.device_type != device_type:
                        risk_score += 50
                    if last_login.ip_address != ip:
                        risk_score += 50
                    if last_login.country != country:
                        risk_score += 100

                LoginActivity.objects.create(
                    user=user,
                    ip_address=ip,
                    user_agent=user_agent,
                    device_type=device_type,
                    country=country
                )

                # ----- OTP Trigger for High Risk -----
                if risk_score >= 100:
                    otp = generate_otp()
                    store_otp(user.id, otp)
                    # send mail asynchronously (Celery recommended)
                    logger.info(f"OTP sent to user {user.email}: {otp}")

            # ----- Security Headers -----
            response["X-Frame-Options"] = "DENY"
            response["X-Content-Type-Options"] = "nosniff"
            response["X-XSS-Protection"] = "1; mode=block"

            # ----- Response Timing -----
            response["X-Response-Time"] = str(time.time() - start_time)

            return response

        except Exception as e:
            logger.exception(f"Middleware Error: {e}")
            return JsonResponse({"error": "Server error"}, status=500)

    async def _get_response(self, request):
        response = self.get_response(request)
        # Support async views
        if callable(getattr(response, "__await__", None)):
            return await response
        return response
```

5️⃣ core/signals.py (Optional login signal)

```
from django.contrib.auth.signals import user_logged_in
from django.dispatch import receiver
from .utils import get_device_type, generate_otp, store_otp, get_country
from .models import LoginActivity

@receiver(user_logged_in)
def login_signal_handler(sender, request, user, **kwargs):
    ip = request.META.get("REMOTE_ADDR")
    user_agent = request.META.get("HTTP_USER_AGENT", "")
    device_type = get_device_type(user_agent)
    country = get_country(ip)

    last_login = LoginActivity.objects.filter(user=user).order_by("-
created_at").first()
    risk_score = 0
    if last_login:
        if last_login.device_type != device_type:
            risk_score += 50
        if last_login.ip_address != ip:
            risk_score += 50
        if last_login.country != country:
            risk_score += 100

    LoginActivity.objects.create(
        user=user,
        ip_address=ip,
        user_agent=user_agent,
        device_type=device_type,
        country=country
    )

    if risk_score >= 100:
        otp = generate_otp()
        store_otp(user.id, otp)
```

6️⃣ settings.py snippet

```
DEBUG = False
ALLOWED_HOSTS = ["yourdomain.com"]
SECRET_KEY = "your-secret-key"

MIDDLEWARE = [
```

```
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'core.middleware.AdvancedSecurityMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
]

# Conditional skip paths
MIDDLEWARE_SKIP_PATHS = ["/static/", "/media/"]

# Redis cache
CACHES = {
    "default": {
        "BACKEND": "django_redis.cache.RedisCache",
        "LOCATION": "redis://127.0.0.1:6379/1",
        "OPTIONS": {"CLIENT_CLASS": "django_redis.client.DefaultClient"},
    }
}

# Logging
LOGGING = {
    "version": 1,
    "handlers": {"console": {"class": "logging.StreamHandler"}},
    "root": {"handlers": ["console"], "level": "INFO"},
}
```

✅ Features of This Template

- Async-ready middleware (supports async views)
- Conditional path skipping
- Device/IP/Country logging
- OTP for suspicious login (risk-based)
- Security headers added automatically
- Full error handling + logging
- Redis cache support for OTP & rate-limiting
- Copy-paste ready, reusable in any Django project

💡 **Next Steps You Can Do:**

1. Integrate Celery for async email sending
2. Connect OTP verification endpoint for users

3. Add Admin Dashboard to view login activity
4. Extend risk scoring for multi-device/multi-location detection

---

যদি চাও, আমি পরের step এ **full OTP verification + Admin Dashboard + GeoIP Map + Risk Scoring** সহ **complete deployable ML API security system** বানিয়ে দিতে পারি।

চাও কি সেটা বানাই?

দারুণ! 🚀 তাহলে আমি তোমার জন্য বানাচ্ছি **"Next Level Django + ML Production Security Master Plan"** — step-by-step roadmap, যা শেষ করলে তুমি পুরোপুরি **enterprise-ready, secure, production-ready Django + ML developer** হয়ে যাবে।

🏆 Next Level Step-by-Step Learning Plan

Step 1: Advanced Middleware Deep Dive (1–2 Days)

Goal: Middleware mastery + async + performance

- Async vs sync middleware
- Lifecycle methods: `__init__`, `__call__`, `process_view`, `process_exception`, `process_template_response`
- Conditional middleware execution
- Request/response modification
- Rate limiting & brute-force protection
- Exception handling in middleware
- Middleware unit testing with `RequestFactory`

Project Practice:

- Upgrade current Advanced Middleware template → add:
  - Multi-device detection
  - API-specific conditional paths
  - Request/response logging with Redis cache

Step 2: Signals & Suspicious Login System (1–2 Days)

Goal: Full login risk monitoring

- Signals: `user_logged_in`, `user_login_failed`
- Risk scoring: Device + IP + GeoIP + Country change
- OTP verification for high-risk login
- Redis caching OTPs
- Email/SMS alerts for suspicious activity
- Admin analytics for login activity

Project Practice:

- Build full **OTP + risk score system** integrated with middleware
- Dashboard for admin → recent suspicious logins, risk scores

---

Step 3: Production-Ready Django Security (2–3 Days)

Goal: Deployable, hardened system

- HTTPS, HSTS, secure cookies, CSRF protection
- Debug=False, strong SECRET_KEY, ALLOWED_HOSTS
- Security headers: `X-Frame-Options`, `X-Content-Type-Options`, `X-XSS-Protection`
- Strong password policies, multi-factor authentication
- Failed login lockouts
- Logging & centralized monitoring (Sentry / ELK)

Project Practice:

- Apply all settings to local + staging → test security headers & cookie behavior
- Try simulated attack → verify middleware blocks / logs correctly

---

Step 4: Redis, Caching & Performance Optimization (1–2 Days)

Goal: Fast, scalable middleware & OTP system

- Redis for: OTP, rate limiting, session cache
- Cache TTL and invalidation
- Avoid DB hits for frequent operations
- Benchmark middleware & API request time

Project Practice:

- Integrate Redis with middleware → OTP + request logs cached

- Measure request-response time → add `X-Response-Time` headers

Step 5: ML API Security & Safe Inference (2–3 Days)

Goal: Secure ML endpoints for production

- File upload validation (type, size, MIME)
- Sandbox / safe inference
- Rate limiting for heavy ML endpoints
- Async ML tasks (Celery)
- Logging & alert for failed inference / errors

Project Practice:

- Build sample video/image ML API → safe & production-ready
- Integrate middleware + signals → track device/IP for ML users

---

Step 6: Admin Dashboard + Analytics (1–2 Days)

Goal: Enterprise-level monitoring

- Track login activity (user, device, IP, country)
- Suspicious login alerts
- OTP verification history
- Graphs/charts for login analytics
- Optional: GeoIP Map for login locations

Project Practice:

- Build admin dashboard in Django Admin or DRF + React
- Display login stats, OTP triggers, and high-risk users

Step 7: Deployment & Hardening (2–3 Days)

Goal: Production-ready server

- Gunicorn + Nginx setup
- SSL / HTTPS / HSTS
- Firewall & fail2ban
- Redis / Celery deployment

- Logging & monitoring integration (Sentry / ELK)
- Backup & disaster recovery plan

Project Practice:

- Deploy Advanced Middleware + ML API on VPS or cloud
- Test: OTP, login detection, GeoIP, ML API rate limiting

Step 8: Optional Advanced Features

- JWT + Device binding system
- Multi-factor authentication (Google Authenticator style)
- Account lock after multiple failed login attempts
- Microservice authentication architecture (for ML-heavy apps)
- Celery + Redis + monitoring alerts

🔥 Outcome After Completing This Plan

- Full understanding of Django middleware (async + production-ready)
- Suspicious login detection & OTP system
- GeoIP + Device/IP monitoring
- Secure ML APIs with validation & rate limiting
- Production-ready server deployment & hardening
- Admin dashboard for analytics & monitoring
- Copy-paste ready reusable template for future projects

💡 **Next Step Recommendation:**

আমি চাইলে তোমার জন্য এই **full roadmap অনুযায়ী একদম ready-to-deploy project template** বানিয়ে দিতে পারি, যেখানে সব feature included থাকবে:

- Advanced Middleware
- Async + Redis
- OTP + Suspicious login
- GeoIP + Device/IP logging
- ML API safe handling
- Production-ready settings + security headers
- Admin dashboard analytics

✅ শুধু copy-paste করলেই production-ready হবে।

চাও আমি সেটা বানাই?